

8.2.6 Zugriff auf Objekte erhalten

In unseren bisherigen Aufgaben konnten wir mithilfe des **Helden** und des **Knappen** alle Level lösen. Doch nicht immer ist der **Zugriff** auf wenige Objekte ausreichend, um das Ziel zu erreichen. Im Sinne der Objektorientierung können wir uns aber Zugriff auf **weitere Objekte** verschaffen, wenn die passenden Methoden gegeben sind.



Aufgabe 1

- a) Öffnen Sie Level 20 und versuchen Sie dieses zu lösen. Erklären Sie knapp, weswegen Ihnen das nicht gelingen kann.

Durch eine kleine „Mogelei“ können Sie das Level dennoch lösen. Betrachten Sie die Objektdiagramme des Zettels mit dem Zauberspruch **zettel** und der Tür **tuer**:

<u>zettel</u>
<pre>x = 2 y = 4 gesammelt = False spruch = "..." #zufall</pre>
<pre>gib_spruch() : String spruch_ausgeben()</pre>

<u>tuer</u>
<pre>x = 0 y = 3 offen = False richtiger_spruch = "..." #zufall</pre>
<pre>ist_passierbar() : Boolean spruch_anwenden(s : str)</pre>

- b) **Nutzen** Sie zunächst das Objekt **zettel**, um den darauf notierten Spruch mit **print(...)** auf der Konsole auszugeben.
- c) **Lösen** Sie das Level, indem Sie den Spruch stattdessen der Tür übergeben.

Tipp:

Sie haben den Helden über die Objekt-Variable **held** bewegt, den Knappen über **knappe** und Methoden mit diesen Objekten aufgerufen, z.B. **knappe.geh()**. Hier müssen Sie jetzt die Objektvariablen **spruch** und **tuer** verwenden sowie die beiden Methoden **gib_spruch()** (für b) und c) sowie **spruch_anwenden(...)** (für c). Speichern Sie sich dazu den erhaltenen Spruch in einer Variable.

Aufgabe 2

Öffnen Sie Level 21 und versuchen Sie mit der gleichen Vorgehensweise das Level zu lösen. Es wird zu einer Fehlermeldung kommen. Notieren Sie die letzte Zeile der Fehlermeldung.

Objekte sind in der Regel **nicht direkt über Variablen referenzierbar**. Stattdessen müssen Sie Zugriff über andere Objekte erhalten, die bereits zur Verfügung stehen. In unserem Fall haben wir immer Zugriff auf das **Level** (vgl. Befehl `level.lade()`) sowie den **Helden**.

Die Objekte für den **Knappen**, die **Tür** und den **Zettel** existieren zwar im Speicher (schließlich werden Sie ja dargestellt), aber Sie besitzen noch keinen Zugriff auf diesen. Um Sie wie gewohnt nutzen zu können, rufen wir sie über Methoden referenzierbarer Objekte (**held**, **level**) ab und binden Sie an eine Variable.

Das kann man beispielhaft so darstellen:

```
???
x = 0
y = 3
offen = False
richtiger_spruch = "" #zufall
ist_passierbar() : Boolean
spruch_anwenden(s : str)
```

`t = objekt.erhalte_tuer()`

```
t
x = 0
y = 3
offen = False
richtiger_spruch = "" #zufall
ist_passierbar() : Boolean
spruch_anwenden(s : str)
```

Aufgabe 3

Betrachten Sie die modifizierten Objektdiagramme des Helden und des Knappen. Diese haben jetzt eine Fähigkeit, um auf **das Objekt direkt vor ihnen** zuzugreifen. Der Held kann zudem jederzeit mit einer seiner Methoden auf das Knappen-Objekt zugreifen. **Lösen** Sie mit den beiden Methoden **Level 21**.

held
x = 2
y = 4
richtung = "S"
geh() : String
ist_auf_herz() : Boolean
...
gib_knappe() : Knappe
gib_objekt_vor_dir(): Objekt

??? (Knappen-Objekt)
x = 0
y = 3
richtung = "S"
geh() : String
ist_auf_herz() : Boolean
...
gib_objekt_vor_dir(): Objekt

Tipp:

Betrachten Sie das Beispiel im Kasten oben, wie ein Objekt **an eine Variable** gebunden werden kann. Nutzen Sie dann das **Helden-Objekt**, um den **Knappen** an eine Variable zu binden (z.B. wieder an **knappen**). Außerdem kann er das Objekt vor sich erkennen (die Tür mit `gib_objekt_vor_dir()`), welche Sie ebenfalls an eine Variable binden müssen. Zuletzt binden Sie mithilfe des Knappen-Objekts den Zettel an eine Variable und können das Level wie Level 20 lösen.

Aufgabe 4

Ergänzen Sie den folgenden Merksatz

Merke:

Anfragen können nicht nur _____ Datentypen zurückliefern, sondern auch höhere Datentypen, wie z.B. _____. Diese können zur weiteren Verwendung auch an _____ gebunden werden.

Übung

Lösen Sie Bonuslevel 104.

??? (Monster-Objekte)
x = 2 y = 3 richtung = "W"
geh() : String ... angriff() gib_objekt_vor_dir(): Objekt